SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Deign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**

Duration : 1 Hour

AT3 - Scenario Based

Code of Conduct:

I **J. SAI DHEERAJ (192311191)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **J. SAI DHEERAJ**

SIMATS ENGINEERING

ASSESSMENT TOOL

1.Given a DFA used for validating a specific PIN pattern bbbc, bca ,c, ca, caaaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

DFA minimization for PIN pattern validation

The given PIN patterns are bbbc, bca, c, ca, and caaaaaa.

A DFA can be designed to accept exactly these five strings over the alphabet \(\Sigma=\{a,b,c\}\). Since the transition diagram or transition table is not provided, the exact equivalent states of a given DFA cannot be identified. However, we can construct a minimized DFA for this finite language.

Step 1: Construct the prefix states

Each state represents the prefix read so far. The accepting states correspond to complete PIN patterns.

- \(q_0\): start state
- \(q_1\): read b
- \(q_2\): read bb
- \(q_3\): read bbb
- \(q_4\): read bbbc (final)
- \(q_5\): read bc (final)
- \(q_6\): read bca (final)
- \(q_7\): read c (final)
- \(q_8\): read ca (final)
- \(q_9\): read caaaa
- \(q_{10}\): read caaaaa
- \(q_{11}\): read caaaaaa (final)
- \(q_d\): dead state for invalid inputs

The transitions follow the characters in each pattern. For example, \(q_0 \xrightarrow{b} q_1\), \(q_1 \xrightarrow{b} q_2\), and \(q_2 \xrightarrow{b} q_3\).

Step 2: Minimized transition table

Let \(A\) be the initial state and \(D\) the dead state.

State	Input a	Input b	Input c
\(\rightarrow A\)	D	B	G
B	D	C	F
C	D	D	H
H	D	D	D
G*	H	D	D
F	E	D	D
E*	D	D	D
I	J	D	D
J	K	D	D
K	F	D	D

The intended transitions for the b-prefix branch are \(A\xrightarrow{b}B\xrightarrow{b}C\xrightarrow{b}H\xrightarrow{c}E\); the bc branch is \(B\xrightarrow{c}F\xrightarrow{a}E\). For the c branch, \(A\xrightarrow{c}G\xrightarrow{a}H\) and the chain of a transitions leads to \(E\). Here, \(E\) is the merged final state for bbbc, bca, and caaaaaa; \(F\) represents the equivalent prefixes bc and caaaaa.

Note: Because the original DFA was not supplied, this is a construction for the stated PIN language, rather than a minimization of a specific given DFA. A complete table should include every state and transition; the prefix-state construction above explains the intended merges.

How state reduction improves embedded systems

1. Less memory: Fewer states require fewer bits to represent the current state.
2. Faster processing: A smaller transition table can reduce lookup and decision overhead.
3. Lower power consumption: Reduced logic and memory accesses can save energy.
4. Lower hardware cost: A simpler circuit may require fewer logic gates.
5. Easier maintenance: The smaller automaton is easier to test and verify.

2.Demonstrate how a given finite automaton can be converted into a regular Expression

The state elimination method converts a finite automaton into a regular expression.

Steps

1. Add a new start state with an ϵ-transition to the original start state.
2. Add a new final state with ϵ-transitions from every original final state.
3. Eliminate intermediate states one by one.
4. When eliminating a state q_k, update the transition between remaining states using:

$$ R_{ij}=R_{ij}+R_{ik}(R_{kk})^*R_{kj} $$

5. Continue until only the new start and final states remain. The expression on the transition between them is the required regular expression.

Example

Consider an automaton that accepts binary strings ending in 01.

Its states track the relevant suffix:

- q_0: initial state
- q_1: the last symbol is 0
- q_2: the string ends in 01 (final state)

Transitions:

- $q_0 \xrightarrow{0} q_1$
- $q_0 \xrightarrow{1} q_0$
- $q_1 \xrightarrow{0} q_1$
- $q_1 \xrightarrow{1} q_2$
- $q_2 \xrightarrow{0} q_1$
- $q_2 \xrightarrow{1} q_0$

The automaton accepts any sequence of 0s and 1s followed by 01.

Therefore, the regular expression is

$$ \boxed{(0+1)^*01} $$

Conclusion: State elimination produces a regular expression equivalent to the language accepted by the finite automaton.

3.Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B)Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar

ambiguous? Justify.

Given:

$$\begin{aligned} S&\rightarrow aSc\mid T\mid U\mid\epsilon\\ T&\rightarrow bTc\mid\epsilon\\ U&\rightarrow aUb\mid\epsilon \end{aligned}$$

A) What is the language defined by the grammar?

The production $S\rightarrow aSc$ adds one a at the beginning and one c at the end.

The nonterminal T generates equal numbers of bs and cs:

$$L(T)=\{b^n c^n\mid n\geq0\}$$

The nonterminal U generates equal numbers of as and bs:

$$L(U)=\{a^n b^n\mid n\geq0\}$$

Hence, the language is

$$\boxed{ L(S)= \{a^i b^j c^{i+j}\mid i,j\geq0\} \cup \{a^{i+j}b^j c^i\mid i,j\geq0\} }$$

This includes the empty string ϵ.

B) Give the derivation tree for the word aaabbc

The word is aaa + bb + c.

Leftmost derivation:

$$\begin{aligned} S&\Rightarrow aSc\\ &\Rightarrow aUc\\ &\Rightarrow aaUbc\\ &\Rightarrow aaaUbbc\\ &\Rightarrow aaabbc \end{aligned}$$

The final step uses $U\rightarrow\epsilon$.

Derivation tree:

S

a

S

c

Replace the inner S with U

U

a

U

b

a

U

b

Yield: aaabbc

C) Is the grammar ambiguous? Justify.

Yes, the grammar is ambiguous.

A grammar is ambiguous if at least one string has two distinct derivations or parse trees.

For the empty string \(\epsilon\), we have three distinct derivations:

1. \(S\Rightarrow\epsilon\)
2. \(S\Rightarrow T\Rightarrow\epsilon\)
3. \(S\Rightarrow U\Rightarrow\epsilon\)

The same string can therefore be derived in multiple ways.

Hence,

\[\boxed{\text{The grammar is ambiguous.}} \]

4.I)Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

I) Show that if L_1 and L_2 are regular, then $L_1\cup L_2$ is also regular.

Proof:

Since L_1 and L_2 are regular, there exist finite automata M_1 and M_2 that accept them.

Construct a new NFA with a new initial state q_s. Add ϵ-transitions from q_s to the initial states of M_1 and M_2.

Keep the final states of both automata as final states.

The new automaton accepts a string if it belongs to L_1 or L_2. Therefore, it accepts exactly their union.

$$\boxed{L_1\cup L_2\text{ is regular}}$$

Hence proved.

II) Is the language $L=\{a^p\mid p\text{ is prime}\}$ regular? Justify.

Answer: No, the language is not regular.

Proof using the pumping lemma:

Assume that L is regular. Let n be its pumping length.

Choose a prime number $p>n$ and consider

$$w=a^p$$

By the pumping lemma, $w=xyz$, where

- $|xy|\leq n$
- $|y|>0$
- $xy^iz\in L$ for every $i\geq0$

Since y contains only as, let $y=a^k$, where $k\geq1$.

Choose $i=p+1$. The length of the resulting string is

$$|xy^{p+1}z|=p+pk=p(1+k)$$

Since p is prime and $k\geq1$, $p(1+k)$ is composite. Thus, the resulting string is not in L, contradicting the pumping lemma.

Therefore,

$$\boxed{\{a^p\mid p\text{ is prime}\}\text{ is not regular}}$$

Hence proved.